Module 2 — Applied Lab: Retail Sales Data Pipeline

> ***Assignment name:** M2-L2-data-pipeline **Repo:** Your `M2-L2-data-pipeline` assignment repo — branch `lab-2/data-pipeline` **Submission:** PR URL from `lab-2/data-pipeline` to `main`*
> ***Grading:** Autograded (CI check) + TA review of PR description*

Objective

Build a complete, modular, tested data pipeline in Python. You will load a real-world-style CSV dataset, clean it (handle missing values and malformed dates), transform it (compute derived features), generate visualizations saved as PNG files, and produce summary statistics. You will also write pytest tests that validate each pipeline stage.

By the end of this lab, you will have a working pipeline that demonstrates the professional standard: modular functions, testable code, reproducible output.

Dataset

Your starter repo includes `data/sales_records.csv` — a synthetic dataset representing Jordanian electronics retail sales records.

Column	Type	Notes
`date`	string	Mostly `YYYY-MM-DD`; some rows use `DD/MM/YYYY`
`store_id`	int	Values 1–5
`product_category`	string	Electronics, Home Appliances, Mobile Accessories, Computing, Audio
`quantity`	float	~5% of rows have missing values
`unit_price`	float	~5% of rows have missing values
`payment_method`	string	Cash, Credit Card, Mobile Payment, Bank Transfer

The messy data is intentional — cleaning it is a core part of the lab.

Getting Started

Accept the assignment: Accept M2-L2-data-pipeline on GitHub Classroom — this creates your personal starter repo with all lab files already in place.

In your repo:

```
git checkout -b lab-2/data-pipeline
```

Your starter files are already in the repo from the Module 2 assignment setup. You should see:

```
pipeline.py              # Function stubs — fill these in
tests/
  test_pipeline.py       # Test stubs — fill these in
  test_lab_autograder.py # CI tests — do not modify
data/
  sales_records.csv      # Your dataset
requirements.txt         # pandas, numpy, matplotlib, pytest
```

Install dependencies (if not already installed from Pre-Work):

```
pip install -r requirements.txt
```

What to Build

`pipeline.py` — 6 Required Functions

Write all 6 functions. Each function stub includes a docstring with the expected input/output and `TODO:` markers. Remove the `TODO:` markers and `pass` statements as you implement each function.

`load_data(filepath)`

Load a CSV file and return it as a DataFrame.

```python
def load_data(filepath):
    """Load sales records from CSV. Returns a DataFrame."""
    # Load the CSV using pd.read_csv
    # Print: "Loaded N records from {filepath}"
    # Return the DataFrame
```

- Use `pd.read_csv(filepath)` — do not hardcode the path inside the function
- Print a progress message: `f"Loaded {len(df)} records from {filepath}"`

`clean_data(df)`

Handle missing values and fix data types. Return a cleaned DataFrame.

```python
def clean_data(df):
    """Handle missing values and fix data types. Returns a cleaned DataFrame."""
    # Fill missing 'quantity' values with the column median
    # Fill missing 'unit_price' values with the column median
    # Parse 'date' to datetime using pd.to_datetime with errors='coerce'
    # Drop rows where BOTH quantity AND unit_price are still missing after fill
    # Print: "Cleaned data: N records"
    # Return the cleaned DataFrame
```

Key requirements:

- Use `df.copy()` at the start — never modify the input in place
- Fill `quantity` NaN with `df['quantity'].median()`
- Fill `unit_price` NaN with `df['unit_price'].median()`
- Parse `date` with `pd.to_datetime(df['date'], errors='coerce')` — this handles the malformatted dates by converting them to `NaT` instead of raising an error
- After cleaning, print a progress message

`add_features(df)`

Add derived columns. Return an enriched DataFrame.

```python
def add_features(df):
    """Add revenue and day_of_week columns. Returns an enriched DataFrame."""
    # Add 'revenue' column = quantity * unit_price
    # Add 'day_of_week' column = day name from the date column
    # Return the enriched DataFrame
```

Key requirements:

- `df['revenue'] = df['quantity'] * df['unit_price']`
- `df['day_of_week'] = df['date'].dt.day_name()` — requires `date` to be datetime type

`generate_summary(df)`

Compute and return a summary statistics dictionary.

```python
def generate_summary(df):
    """Compute summary statistics. Returns a dict."""
    # Return a dict with:
    #   'total_revenue': sum of all revenue values
    #   'avg_order_value': mean of all revenue values
    #   'top_category': product_category with highest total revenue
    #   'record_count': number of records in df
```

Key requirements:

- Use `df.groupby('product_category')['revenue'].sum().idxmax()` for top category
- Return a Python `dict` — not a DataFrame

`create_visualizations(df, output_dir='output')`

Generate 3 charts and save them as PNG files.

```python
def create_visualizations(df, output_dir='output'):
    """Create and save 3 charts to output_dir."""
    # Create output directory with os.makedirs(output_dir, exist_ok=True)
    # Chart 1: Bar chart — total revenue by product category
    # Chart 2: Line chart — daily revenue trend (aggregate by date)
    # Chart 3: Horizontal bar chart — average order value by payment method
    # Save each chart as a PNG with fig.savefig() — do NOT use plt.show()
    # Close each figure with plt.close(fig) after saving
```

Key requirements:

- `os.makedirs(output_dir, exist_ok=True)` — create the directory if it doesn’t exist
- Use `fig, ax = plt.subplots()` pattern for each chart
- Save each figure: `fig.savefig(f'{output_dir}/chart_name.png', dpi=150, bbox_inches='tight')`
- Call `plt.close(fig)` after each save to free memory
- **Do not use `plt.show()`** — this blocks execution in pipeline scripts

Suggested file names:

- `output/revenue_by_category.png`
- `output/daily_revenue_trend.png`
- `output/avg_order_by_payment.png`

`main()`

Orchestrate the full pipeline.

```python
def main():
    """Run the full pipeline end-to-end."""
    # Call load_data → clean_data → add_features → generate_summary
    # Print the summary statistics
    # Call create_visualizations
    # Print "Pipeline complete."

if __name__ == "__main__":
    main()
```

The `if __name__ == "__main__"` guard is required — it prevents the pipeline from running when `pipeline.py` is imported during testing.

`tests/test_pipeline.py` — At Least 3 Test Functions

Your starter file has test stubs. Implement at least these 3 test functions:

`test_load_data_returns_dataframe`

```python
def test_load_data_returns_dataframe():
    """load_data returns a DataFrame with expected columns."""
    # Load the CSV
    # Assert the result is a pandas DataFrame
    # Assert len(df) > 0
    # Assert expected columns are present: 'date', 'store_id', 'product_category'
```

`test_clean_data_no_nulls`

```python
def test_clean_data_no_nulls():
    """After clean_data, quantity and unit_price have no NaN values."""
    # Load and clean the data
    # Assert cleaned['quantity'].isna().sum() == 0
    # Assert cleaned['unit_price'].isna().sum() == 0
```

`test_add_features_creates_revenue`

```python
def test_add_features_creates_revenue():
    """add_features creates a 'revenue' column equal to quantity * unit_price."""
    # Load, clean, and add features
    # Assert 'revenue' column exists in the result
    # Assert the revenue values equal quantity * unit_price
    # Use pd.testing.assert_series_equal for float comparison
```

Running the Pipeline

```
# Run the full pipeline
python pipeline.py

# Expected output:
# Loaded 500 records from data/sales_records.csv
# Cleaned data: ~487 records
# === Summary ===
# Total Revenue: ...
# Average Order Value: ...
# Top Category: ...
# Record Count: ...
# Pipeline complete.

# Three PNG files appear in output/
```

Running Your Tests

```
pytest tests/test_pipeline.py -v
```

All 3 (or more) tests must pass before you submit.

Committing and Submitting

When your pipeline runs end-to-end and your tests pass:

```
# Stage your files
git add pipeline.py tests/test_pipeline.py output/

# Commit
git commit -m "Lab 2: complete data pipeline with tests"

# Push
git push -u origin lab-2/data-pipeline
```

Open a PR from `lab-2/data-pipeline` to `main` in your assignment repo.

Your PR description must include:

1. The summary statistics output (copy-paste from your terminal)
2. Confirmation that `pytest tests/test_pipeline.py -v` passes (copy-paste output or “all N tests passed”)
3. Paste your PR URL into TalentLMS → Module 2 → Lab 2 to submit this assignment.

What the Autograder Checks

The CI workflow runs automatically when you open a PR. It checks:

Test	What it validates
`test_pipeline_file_exists`	`pipeline.py` is present
`test_pipeline_runs_end_to_end`	`python pipeline.py` exits with code 0
`test_output_charts_created`	At least 3 PNG files exist in `output/`
`test_summary_output_contains_revenue`	stdout contains summary statistics
`test_learner_tests_pass`	Your `tests/test_pipeline.py` tests all pass
`test_pipeline_functions_importable`	`load_data`, `clean_data`, `add_features`, `generate_summary` are importable

If any check fails, click the red X on the Actions tab for details. Fix the issue, commit, and push — the autograder re-runs automatically.

Common Mistakes

Mistake	Fix
Hardcoding the file path inside `load_data()`	Use the `filepath` parameter
Forgetting `os.makedirs('output', exist_ok=True)`	Calling `savefig()` on a missing directory raises `FileNotFoundError`
Using `plt.show()` instead of `fig.savefig()`	`plt.show()` blocks execution or saves a blank figure
Using `plt.show()` before `fig.savefig()`	The figure is cleared before saving — output is blank
Not using `pd.to_datetime(errors='coerce')`	Malformatted dates raise a `ValueError`
Missing `if __name__ == "__main__"` guard	`main()` runs when the file is imported during testing
Tests import from wrong module name	Use `from pipeline import ...` not `from data_pipeline import ...`

Challenge Tiers

These optional challenges deepen the lab work. They are **not graded** — the base assignment above is your graded deliverable. Tiers build on each other in complexity. No additional starter files are needed; extend your existing `pipeline.py` and test files.

Challenge 1 — Edge-Case Engineering

Make your pipeline robust to adversarial input. Write a pytest test for each of these edge cases, then update your pipeline functions so all tests pass:

1. **Empty CSV** — a file with headers but zero data rows. Your pipeline should handle this gracefully (log a warning, produce an empty summary) without crashing.
2. **All-null column** — a CSV where the entire `quantity` column is NaN. `median()` returns NaN, and `groupby().idxmax()` on empty groups throws. Your pipeline should detect and handle this.
3. **Single-row CSV** — a CSV with exactly one data row. Aggregations like `groupby().sum()` still need to work, and charts should render without errors.

Why this is harder than it sounds: `median()` on an all-null column returns NaN (silently). `groupby(...).idxmax()` on an empty DataFrame raises a `ValueError`. `day_name()` on `NaT` raises in some Pandas versions. Each edge case exposes a different category of defensive coding.

What to produce:

- Updated `pipeline.py` that handles all three cases
- At least 3 new tests in `tests/test_pipeline.py` (one per edge case)

Challenge 2 — Parameterized Pipeline

Refactor your pipeline to be driven by a JSON configuration file instead of hardcoded values. Your config file should specify:

- Input file path
- Output directory
- Which columns to fill missing values for (and fill strategy: median, mean, or zero)
- Which column to group by for summary statistics
- Which charts to generate (list of chart types)

Then find or create a second dataset (different domain — e.g., weather station readings, restaurant inspections, library checkouts) and write a second config file for it. **Demonstrate that your pipeline produces correct output on both datasets without changing any Python code** — only the config changes.

Engineering pattern: Separation of logic from configuration. This pattern is fundamental to production data systems.

Notes:

- Use JSON for the config file (stdlib `json` module — no extra packages needed). YAML is not bundled in M2 — stick with JSON for this stretch.
- The `main()` function should accept an optional config path argument.

Challenge 3 — Data Validation Framework

Build a `validate.py` module that defines data quality expectations declaratively and runs them before and after cleaning. This is a mini version of what tools like Great Expectations and Pandera do in production.

Your validation framework should:

1. Let you define expectations as a list of rules — for example:
    - “Column `quantity` should have no more than 10% null values”
    - “Column `unit_price` values should be between 0 and 10000”
    - “Column `date` should have no duplicate values within the same `store_id`”
2. Run all expectations against a DataFrame and produce a validation report (pass/fail per expectation, with counts and details)
3. Run validation **twice** — once on the raw data (documenting what’s wrong) and once on the cleaned data (confirming the issues are resolved)
4. Output the report as a structured format (JSON or printed table)

Design challenge: The hard part is the API design, not the individual checks. How do you represent “column X has no nulls” and “column Y values between A and B” in the same data structure? How do you make it extensible without overengineering?

What to produce:

- `validate.py` with your validation framework
- A set of expectations defined for `sales_records.csv`
- A before/after validation report showing raw data failures and post-cleaning passes

---

Module 2 of 12 — Applied AI & ML Systems

© 2026 LevelUp Economy. All rights reserved. Unauthorized reproduction or distribution of this material is prohibited.